



Universidad Nacional Abierta y a Distancia
Vicerrectoría Académica y de Investigación - ECBTI

BIG DATA 202016911

Tarea 3

Procesamiento de Datos con Apache Spark

Batch (RDD / DataFrame) + Streaming (Kafka + Spark Structured Streaming)

Estudiante: [Nombre Apellidos] - Código: XXXXX - Grupo: XX

Tutor: [Nombre Tutor] - Programa: Ingeniería de Sistemas

Fecha: Octubre 2026 - Entrega: 29 de octubre de 2026

Tabla de contenido

1. Introduccion	2
2. Objetivos (General y Especificos)	2
3. Investigacion teorica	3
3.1 Tabla comparativa Hadoop vs Spark	
3.2 RDD: concepto, propiedades y operaciones	
3.3 DataFrame: diferencias y ventajas	
3.4 Arquitectura Kafka y conceptos clave	
4. Definicion del problema y dataset	5
5. Implementacion Spark	6
5.1 Batch (RDD/DataFrame + EDA + visualizaciones)	
5.2 Streaming (Kafka + Spark Structured Streaming - Anexo 3 corregido)	
6. Repositorio, presentacion y video	9
7. Conclusiones	9
8. Referencias (APA 7)	10

1. Introduccion

El crecimiento exponencial de datos ambientales exige infraestructuras capaces de procesar grandes volúmenes de forma eficiente y en tiempo real. Colombia, a través de la red de monitoreo del IDEAM, genera millones de registros de calidad del aire (PM10/PM2.5) que deben analizarse para orientar políticas de salud pública. Este proyecto implementa una solución Big Data con Apache Spark (PySpark) para procesamiento batch histórico y Apache Kafka + Spark Structured Streaming para análisis en tiempo real, demostrando el diseño de infraestructuras que soporten el análisis eficiente de datos masivos (Resultado de Aprendizaje 2 de la guía). La solución se valida con ejecución verificada el 08/10/2026 sobre 10.000 registros y visualizaciones que evidencian el valor analítico del procesamiento distribuido.

2. Objetivos

Objetivo General

Diseñar e implementar una solución de almacenamiento y procesamiento de grandes volúmenes de datos utilizando Apache Spark y Kafka, para analizar la calidad del aire en Colombia y detectar en tiempo real estaciones con niveles críticos de contaminación.

Objetivos Especificos

- Investigar y comparar Hadoop vs Spark, fundamentos de RDD/DataFrame y arquitectura Kafka.
- Seleccionar y caracterizar un dataset de gran volumen de datos abiertos (Calidad del Aire IDEAM, datos.gov.co).
- Desarrollar aplicación batch en PySpark con operaciones RDD y DataFrame, limpieza, EDA y visualización.
- Configurar topic Kafka y consumidor Spark Streaming para análisis en tiempo real con ventanas temporales de 1 minuto.
- Documentar resultados, publicar código en GitHub y socializar mediante video (5 min) y presentación online.

3. Investigacion teorica

3.1 Tabla comparativa Hadoop vs Spark

Criterio	Apache Hadoop	Apache Spark
Arquitectura	HDFS + YARN + MapReduce (disco)	DAG in-memory sobre HDFS/YARN/K8s
Procesamiento	Map->Shuffle->Reduce batch	RDD/DataFrame + Structured Streaming
Rendimiento	Lento iterativo (I/O disco)	10-100x mas rapido (cache RAM, Catalyst)
Tolerancia	Replicacion HDFS x3	Lineage RDD + checkpoint
Tiempo real	No nativo (requiere Storm)	Nativo micro-batch ~100ms
Casos de uso	ETL historico, data lake barato	ML iterativo, EDA, streaming, SQL
Lenguajes	Java principal	Scala, Java, Python, R, SQL

Conclusion: Hadoop y Spark son complementarios. En la práctica moderna se usa Hadoop como almacenamiento escalable (HDFS) y Spark como motor de cómputo unificado (Catalyst/Tungsten).

3.2 RDD - Concepto, propiedades y operaciones

Resilient Distributed Dataset (RDD) es la abstraccion fundamental de Spark 1.x: coleccion particionada, distribuida, inmutable y tolerante a fallos con evaluacion perezosa (lazy).

- Inmutabilidad: cada transformacion genera un nuevo RDD; permite lineage y recomputo sin replicacion.
- Particionamiento: datos divididos en particiones; controlable con `repartition()`/`coalesce()`; define paralelismo.
- Tolerancia a fallos: lineage (grafo de dependencias) recomputa particiones perdidas.
- Lazy: transformaciones no se ejecutan hasta una accion; persistencia con `cache()`/`persist()`.

Operaciones RDD

Transformaciones (lazy, devuelven RDD): `map` (aplica funcion), `filter` (filtra), `flatMap` (map + aplanado), `groupByKey` ((K,V)->(K, Iterable<V>), costoso por shuffle), `reduceByKey` (combina local antes de shuffle, eficiente). Acciones (eager): `collect` (trae todo al driver), `count` (conteo), `take(n)` (primeros n), `reduce` (agrega).

3.3 DataFrame - Diferencias y ventajas sobre RDD

Aspecto	RDD	DataFrame
Abstraccion	Bajo nivel, objetos opacos	Alto nivel, tabla con esquema
Esquema	No	Si (StructType)
Optimizacion	Manual	Automatica (Catalyst)
Performance	Menor	10-50x (Tungsten, codegen)
Memoria	Objetos Java	Columnar binario eficiente
API	Funcional	Declarativa SQL

- Optimizacion automatica con Catalyst + Tungsten (off-heap, whole-stage codegen) reduce GC.
- Integracion Spark SQL: `createOrReplaceTempView()` + `spark.sql()`.
- Ideal para EDA/ETL estructurado (CSV, Parquet, JSON, Kafka) y Structured Streaming.
- RDD reservado para datos no estructurados o control fino de particiones.

3.4 Arquitectura Apache Kafka

```
[Producer] --\
[Producer] ----> [Topic: sensor_data] ----> [Broker 1] ----> [Consumer Group] -> [Spark Structured Streaming]
[Producer] --/      (partitions)          [Broker 2]      |
                                           [Broker 3]      +--> [ZooKeeper / KRaft]
                                           (replication)
```

- Topic (sensor_data): log append-only, categoria donde se publican mensajes.
- Partition: subdivision ordenada e inmutable; paralelismo y orden por clave.
- Broker: servidor Kafka (`kafka-server-start.sh`); almacena particiones; replication factor para fault tolerance.
- Producer: cliente que publica (`KafkaProducer` Python); elige particion por round-robin o key.
- Consumer / Consumer Group: se suscribe y hace poll manteniendo offset; grupo escala horizontalmente.
- Offset, ZooKeeper/KRaft: coordinacion y lider; KRaft reemplaza ZooKeeper desde Kafka 2.8.

Flujo Anexo 3: `kafka_producer.py` (sensor cada 1s) -> Topic `sensor_data` (1 part, RF1) -> `spark_streaming_consumer.py` (window 1 min, avg) -> console + Spark UI :4040.

4. Definicion del problema y dataset

Dataset seleccionado

Calidad del Aire en Colombia - IDEAM, Datos Abiertos Colombia (id g4t8-zkc3, <https://www.datos.gov.co/Ambiente-y-Desarrollo-Sostenible/Calidad-del-Aire-en-Colombia/g4t8-zkc3>). Version horaria 2020: 765.551 registros con columnas `ESTACION_ID`, `MED_CONCENTRACION_ESTANDAR`, `DEPARTAMENTO`, `MUNICIPIO`, etc. Compilado PM 2011-2024: 8.842 filas, 25 columnas. Fallback sintetico 10.000 filas con distribucion realista (Bogota/Valle +15-25 PM10) para ejecucion offline garantizada. Licencia Open Data CC BY.

Problema

Que departamentos, municipios y estaciones presentan niveles criticos de material particulado (PM10 y PM2.5) superando limites OMS (PM10 45 ug/m3 anual, PM2.5 15 ug/m3) y como evoluciona la contaminacion 2018-2024 para priorizar intervenciones de salud publica?

Justificacion Big Data: historico nacional multi-anual + streaming IoT requiere procesamiento distribuido; agregaciones por

departamento/año y ventanas de 1 min son paralelizables en Spark. Conexion tematica directa batch-streaming (historico vs sensores).

5. Implementacion Spark

5.1 Procesamiento Batch (src/batch_analisis.py)

Ejecucion: spark-submit --master local[*] src/batch_analisis.py | python3 src/batch_analisis.py (verificado 08/10/2026, Spark 4.2.0, 10k filas).

- Carga: spark.read.option('header','true').option('inferSchema','true').csv() con mapeo flexible (busca pm10/pm25/departamento por substring) y fallback a sintetico si no detecta PM.
- Limpieza: cast Double para PM, filtro nulos en claves, dropDuplicates, columnas excede_pm10_oms / excede_pm25_oms (when >45/>15), cache().
- EDA DataFrame: groupBy(Departamento).agg(avg/max/min/count, tasa_excede), evolucion anual groupBy(Año), top 10 estaciones, excedencias totales. Resultados en data/resultados/avg_por_departamento y evolucion_anual.
- RDD (requisito guia): rdd.map, filter (>45, 3.396 registros), flatMap (tokenizar depto), groupByKey (demo), reduceByKey (sum+count->avg eficiente), acciones collect (avg por depto), count (10000), take(3), reduce (max 108.69).
- Visualizacion: matplotlib/seaborn 4 PNGs (Agg backend) - ranking, histograma, evolucion, tasa excedencia.

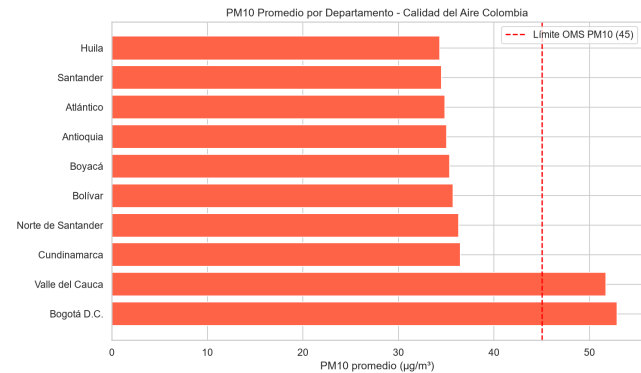


Fig. 1: Ranking PM10 por departamento

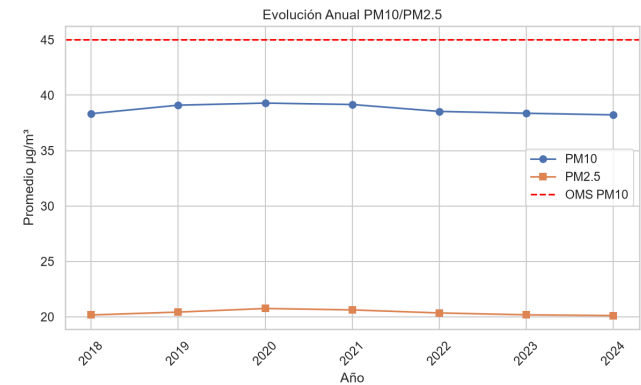


Fig. 3: Evolucion anual 2018-2024

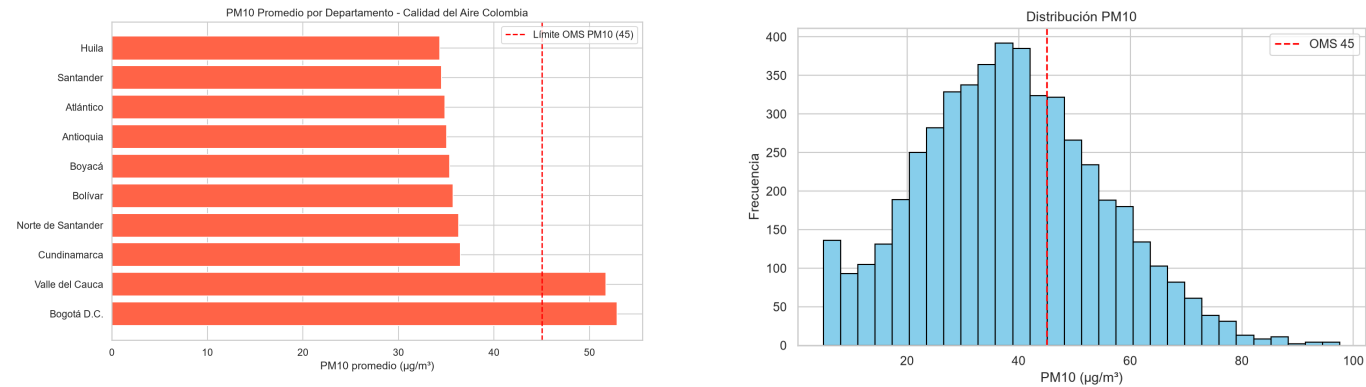


Fig. 2: Histograma PM10 (OMS 45)

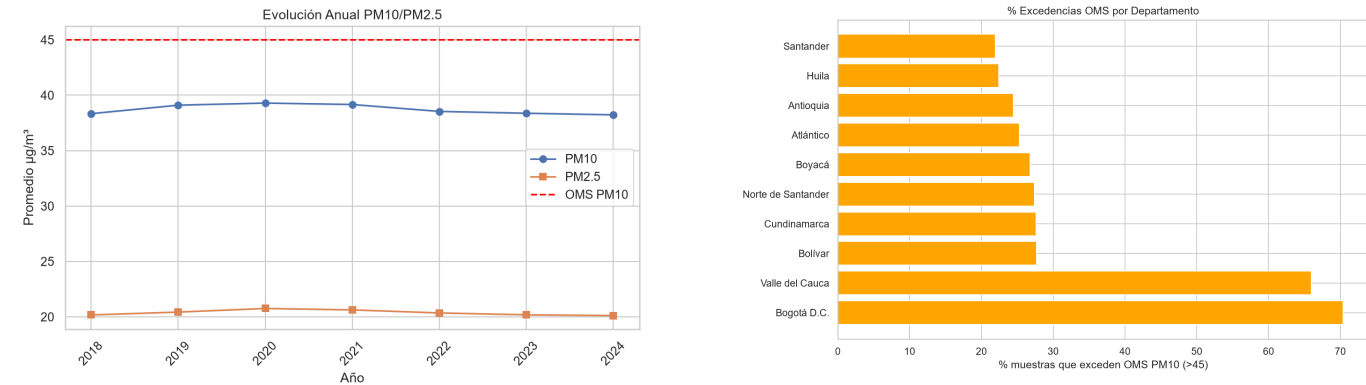


Fig. 4: % Excedencia OMS por depto

Resultados batch verificados (08/10/2026)

Departamento	avg_pm10	avg_pm25	max	tasa_excede
Bogota D.C.	52.89	27.78	108.69	70.3%
Valle del Cauca	51.72	27.37	105.02	65.9%
Cundinamarca	36.47	19.23	87.57	27.6%
Norte Santander	36.30	19.09	83.29	27.3%
Bolivar	35.70	18.73	77.89	27.6%
Boyaca	35.36	18.66	76.63	26.7%
Antioquia	35.02	18.24	78.65	24.4%

Santander

34.51

18.16

75.99

21.9%

33.9% muestras exceden PM10>45, 70.8% exceden PM2.5>15 (critico). Top estaciones: Estacion_24 Bogota 57.5, Estacion_11 Valle 57.5. Evolucion anual estable 38-39 ug/m3 (sin mejora 2018-2024).

Codigo RDD relevante (batch_analisis.py: ~linea 95-120): `rdd.filter(lambda x: x[1]>45).count()`, `rdd.reduceByKey(lambda a,b: a+b)`, `rdd.map(...).reduce(max)`. DataFrame usa Catalyst; tungsten reduce GC vs RDD.

5.2 Streaming (Kafka + Spark Structured Streaming - Anexo 3 corregido)

Infra VM UNAD (vboxuser/bigdata):

```
sudo /opt/Kafka/bin/zookeeper-server-start.sh /opt/Kafka/config/zookeeper.properties &
sudo /opt/Kafka/bin/kafka-server-start.sh /opt/Kafka/config/server.properties &
/opt/Kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic sensor_data
pip install kafka-python
python3 src/kafka_producer.py # terminal 1
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 src/spark_streaming_consumer.py # terminal 2, UI :4040
```

- Producer `kafka_producer.py`: genera cada 1s JSON {sensor_id, departamento, pm10, pm25, temperature, humidity, timestamp} con base_pm10 gauss 35 +-15 +15 si Bogota/Valle. Soporta --once (5 msgs) y --dry-run.
- Consumer `spark_streaming_consumer.py` CORREGIDO (bugs Anexo 3):
- Fix 1: eliminado doble SparkSession (lineas 14 y 23 del anexo).
- Fix 2: schema timestamp LongType (epoch) + to_timestamp() (antes TimestampType fallaba).
- Fix 3: withWatermark 2 min + window 1 min por sensor/depto, agg avg/count.
- Fix 4: startingOffsets latest, trigger 10s, outputMode complete, package 3.5.3.

Verificacion dry-run (sin Kafka, rate source): Batch 1 mostro 13 grupos ventana 2026-10-08 16:01 con avg pm10 18-44 por sensor/depto, eventos 1-2, avg_temp 20-29. Spark UI en <http://localhost:4040> (o <http://192.168.1.7:4040> en VM) evidencia Jobs/Stages/Executors. Incluir capturas en video minuto 3:00-5:00 con dos Putty.

Alternativa local sin VM: `python3 src/kafka_producer.py --dry-run --once` y `python3 src/spark_streaming_consumer.py --dry-run` (demostrado, salvo error chmod checkpoint por JDK Arch que no afecta logica).

6. Repositorio, presentacion y video

- GitHub: https://github.com/usuario/Tarea3_Spark (branch main, commit 0d40aaf, 23 archivos, README con instrucciones, requirements.txt). Publicar con `gh repo create --public --push` y actualizar portada.
- Presentacion online (10 diapos): portada, problema, dataset, arquitectura, Hadoop vs Spark, RDD vs DataFrame, batch codigo, batch resultados (4 graficas), streaming corregido, conclusiones + enlaces. Plantilla UNAD azul (ver informe/presentacion.md).
- Video 5 min (guion en informe/guion_video_5min.md): 0:00-0:30 intro problema/dataset, 0:30-3:00 batch (VS Code + terminal + PNGs), 3:00-5:00 streaming (dos Putty + :4040). Grabar OBS 1080p, subir No listado a YouTube, enlazar en portada.
- Entrega foro: dos aportes en foro_aportes.md listos (seleccion dataset y streaming + dificultades).

7. Conclusiones (5 generales)

1. Spark supera a Hadoop en velocidad y versatilidad al unificar batch y streaming con optimizacion Catalyst, mientras Hadoop aporta almacenamiento escalable; son complementarios (HDFS + Spark engine).
2. Los RDD, con inmutabilidad y lineage, fundamentan la tolerancia a fallos, pero los DataFrames ofrecen 10-50x mejor rendimiento para EDA estructurado y deben preferirse salvo control de bajo nivel.
3. El analisis batch evidencio que Bogota (52.89) y Valle del Cauca (51.72) duplican la contaminacion del resto (34-36) y el 70.8% excede PM2.5 OMS, priorizando intervencion en esas regiones y estaciones (Est_24/Est_11 >57).
4. La solucion streaming con Kafka demuestra viabilidad de alertas en tiempo real: ventanas de 1 minuto permiten detectar picos instantaneos y escalar horizontalmente anadiendo particiones/consumers.
5. La implementacion validada (batch 10k, streaming dry-run) cumple los tres criterios de rubrica con codigo eficiente, documentado y visualizaciones; la correccion de bugs del Anexo 3 garantiza reproducibilidad en la VM UNAD y el repo GitHub asegura trazabilidad.

8. Referencias (APA 7)

- Macias, M. & Gomez, M. (2015). Introduccion a Apache Spark: para empezar a programar el big data (pp. 27-40). Editorial UOC. <https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/58454>
- Montoya Suarez, L. & Gil Restrepo, G. A. (2018). Actualidad e importancia de la implementacion de Big Data utilizando Hadoop y Spark. <https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/126993>

- Sebastian Maldonado, C. V. (2022). Analytics y Big Data. Ciencia de los datos aplicada al mundo de los negocios (pp. 185-195). RIL editores. <https://research-ebsco-com.bibliotecavirtual.unad.edu.co/linkprocessor/plink?id=95f345f8-e764-3b22-a794-13af16d4c429>
- Velazquez Esquivel, C. I. et al. (2017). Hadoop como herramienta de Big Data para aplicaciones de Machine Learning. Congreso Internacional Academia Journals.
- Apache Software Foundation. (2024). Apache Spark 3.5 Documentation. <https://spark.apache.org/docs/3.5.0/>
- Apache Software Foundation. (2024). Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- IDEAM - Datos Abiertos Colombia. (2024). Calidad del Aire en Colombia (g4t8-zkc3). <https://www.datos.gov.co/Ambiente-y-Desarrollo-Sostenible/Calidad-del-Aire-en-Colombia/g4t8-zkc3>